QUANTMIND

Product Overview · Architecture · Technical Reference

Mobile App · Web / Marketing · Backend Infrastructure · Management Dashboard

Version 2.0 · March 2026 · Confidential

0

Professional Portfolio Risk Intelligence Platform

"Most investing apps help you buy assets. Quantmind helps you understand risk before you regret it."

Quantmind is an advanced cross-platform mobile application designed to help investors, students, and financial analysts understand portfolio risk and uncertainty through large-scale stochastic simulation. The platform transforms uncertainty into actionable insight by leveraging Monte Carlo simulation combined with Geometric Brownian Motion (GBM) to generate thousands of possible future price paths for multiple assets within a portfolio, offering a realistic view of how investments may evolve over time under uncertain market conditions.

Quantmind bridges the gap between academic quantitative finance and practical investing tools by delivering professional-grade simulation capabilities in a simple, mobile-first experience — empowering users to make more informed, data-driven investment decisions.

Core Value Proposition

Pillar	Description
Academic Rigor	Professional-grade quantitative methods: GBM, Cholesky, Monte Carlo, jump diffusion
Mobile Accessibility	Institutional-grade simulation tools in a pocket-sized, cross-platform experience
AI Augmentation	Multi-model AI intelligence — task routing, execution, analytics, and guided interpretation
Web Dashboard	Centralized app management, OTA updates, feature flags, and release governance
Educational Value	Learning integrated with practical application for students and beginners
Real-Time Relevance	Live market connection, monitoring, and sensitivity analysis

Platform Architecture Summary

Pillar	Stack	Deploy	Owner
Mobile App	React Native · Expo · Zustand · RN Skia	EAS Build → App Store / Play	Mobile team
Web / Marketing	Next.js 14 · Tailwind · Framer Motion	Vercel (CDN edge)	Web team
Backend	Supabase · FastAPI · Python · Deno	Railway · Supabase Cloud	Backend team
Dashboard	Next.js 14 · Recharts · NextAuth	Vercel (private)	Platform team

Simulation Engine

Monte Carlo Framework — up to 10,000 simulation paths per portfolio

- Full probability distribution of future portfolio values
- Statistically rigorous uncertainty quantification with configurable confidence intervals

Geometric Brownian Motion (GBM) — realistic asset price evolution under uncertainty

- Incorporates volatility (σ) and drift (μ) parameters
- Supports long-term multi-asset forecasting

Correlation Modeling & Cholesky Decomposition

- Models asset co-movement using user-defined correlation matrices
- Transforms independent random variables into correlated ones for realistic multi-asset paths

Metric	Description
Value at Risk (VaR)	Quantifies potential losses at 95% and 99% confidence levels
Conditional VaR (CVaR)	Expected shortfall beyond the VaR threshold
Probability of Loss	Calculates likelihood of negative returns over the horizon
Expected Return	Mean expected portfolio performance across all simulation paths
Sharpe / Sortino Ratios	Risk-adjusted return and downside deviation focus metrics
Maximum Drawdown	Peak-to-trough decline across simulated trajectories
Diversification Score	Composite 0–100 rating based on HHI, avg correlation, sector spread
Tail Risk Metrics	Measures extreme downside scenarios beyond standard distributions

01

React Native · Expo · iOS & Android

The Quantmind mobile app is the primary user-facing product. It delivers professional-grade Monte Carlo portfolio simulation, multi-model AI assistance, real-time risk analytics, and subscription management through a single cross-platform codebase built with React Native and the Expo managed workflow.

Layer	Technology
Framework	React Native 0.74 + Expo SDK 51 — managed workflow, EAS Build for production binaries
Language	TypeScript 5.4 strict mode throughout — no implicit any, no loose types
Navigation	React Navigation v6 — stack + bottom tabs + modals. Type-safe route params via typed navigation props
State Management	Zustand 4 — authStore, portfolioStore, simulationStore, chatStore, subscriptionStore
Data Fetching	TanStack Query v5 — server state, background refresh, optimistic updates, infinite queries for history
Charts	React Native Skia (GPU-accelerated) for fan charts, histograms, correlation heatmaps, gauges
Animations	Reanimated 3 + Gesture Handler — worklet-based 60fps animations, spring physics
Networking	Supabase JS SDK v2 — typed queries, realtime subscriptions, RLS-aware client
Subscriptions	RevenueCat SDK — purchase flow, entitlement checking, receipt validation
Monitoring	Sentry React Native — crash reporting, performance tracing, breadcrumbs
Testing	Jest + React Native Testing Library (unit); Maestro (E2E on device)
OTA Updates	Expo Updates — EAS Update channel managed via web dashboard

1.2 Application Architecture

1.2.1 Folder Structure

```
apps/mobile/
├── src/
│   ├── screens/         # Feature-grouped screen components
│   │   ├── auth/        # Login, Signup, ForgotPassword, Onboarding
│   │   ├── portfolio/   # PortfolioList, Builder, AssetSearch, Detail
│   │   ├── simulation/  # Config, Results, PathViewer, ExportSheet
│   │   ├── ai/          # ChatScreen, PortfolioDoctor, GoalCheck, WhatIf
│   │   ├── analytics/   # Diversification, Heatmap, RiskGauge, StressTest
│   │   ├── subscription/ # Paywall, UpgradeSheet, BillingHistory
│   │   └── settings/    # Profile, Notifications, Security, Support
│   ├── navigation/      # RootNavigator, AuthStack, AppStack, types.ts
│   ├── store/           # Zustand slices
│   ├── services/        # API clients and adapters
│   ├── hooks/           # useSimulation, useEntitlement, useMarketData...
│   ├── components/      # Reusable UI: charts/, metrics/, ai/, shared/
│   ├── theme/           # FX1 design tokens, colours, typography, spacing
│   └── utils/           # formatters, validators, haptics, crypto helpers
├── app.json             # Expo config: scheme, splash, icon, permissions
├── eas.json             # EAS Build profiles: dev / preview / production
└── metro.config.js      # Module resolver, SVG transformer
```

1.2.2 State Architecture

Store	Responsibilities
authStore	user, session, jwt, signIn(), signUp(), signOut(scope), refreshSession(), isAdmin
portfolioStore	portfolios[], activePortfolioId, CRUD actions, optimistic local mutations synced to Supabase
simulationStore	results{}, status (idle|running|streaming|done|error), history[], runSimulation(), cancelJob()
chatStore	messages[], streamingMessageId, send(), clear(), retryLast(), modelIndicator (haiku|sonnet|opus)
subscriptionStore	tier, entitlements{}, trialDaysRemaining, refresh(), checkFeature(featureKey) → boolean
settingsStore	notificationPrefs, themeMode, expertiseLevel, privacyConsents, biometricEnabled

1.2.3 Service Layer

Service	Responsibility
supabase.ts	Singleton Supabase JS client. Auth state listener triggers store hydration. Typed schema via generated database.types.ts
simulationApi.ts	POST /simulate — builds SimulationRequest, polls /simulate-status/:jobId at 500ms interval. Cancellable via AbortController.
portfolioApi.ts	CRUD wrappers for portfolios Edge Function. Optimistic updates: local state updated immediately, server sync in background.
marketDataApi.ts	Ticker search (debounced 300ms), historical prices, correlation matrix computation. Results cached 1hr in TanStack Query.
claudeService.ts	Streaming AI chat via /ai-chat Edge Function. Uses ReadableStream reader to deliver tokens to chatStore incrementally.
revenueCatService.ts	purchase(packageId), restorePurchases(), checkEntitlement(id), getOfferings(). Wrapped to throw typed errors for the UI.
analyticsService.ts	track(eventName, props) dispatched to analytics_events via Edge Function. Batched every 5s or on app background.

1.3.1 Simulation Flow — Client Side

Step	Description
Step 1 — Validation	Client validates portfolio: weights sum to 1.0 ±0.01, all tickers non-empty, correlation matrix positive semi-definite (Cholesky pre-check in TS).
Step 2 — Request	POST /simulate with SimulationRequest payload. Receives {jobId, estimatedDuration}. simulationStore sets status='running'.
Step 3 — Polling	Poll GET /simulate-status/:jobId every 500ms. Response: {status: 'pending'|'running'|'done'|'failed', progress: 0-100, partialResult?}
Step 4 — Streaming	On status='running', partialResult streams simulation batches — fan chart draws incrementally in React Native Skia.
Step 5 — Completion	On status='done', full SimulationResult stored in simulationStore and persisted to simulations table via Supabase.

Step	Description
Step 6 — Display	SimulationResultsScreen renders FanChart, ProbabilityHistogram, MetricsGrid, RiskGauge with entry animations.

1.3.2 Chart Components — Technical Spec

Component	Implementation	Data Contract
FanChart.tsx	React Native Skia — draws 5 percentile paths as Bézier curves. Animated draw-on via Reanimated progress value (0→1, 800ms). Touch: scrub vertical cursor.	paths: PercentilePaths, startDate: Date, horizon: number, varThreshold?: number
ProbabilityHistogram.tsx	Skia Path — computes N bins from terminal values array. Bins left of 0 filled Red, right Teal. Animated via staggered bar height reveal.	terminalValues: number[], numBins?: number, currentValue: number
CorrelationHeatmap.tsx	Skia Grid — N×N cells. Color interpolation Red→Neutral→Teal per value. Highlight cells |corr|>0.85 with Pink border pulse animation.	matrix: number[][], tickers: string[]
DiversificationGauge.tsx	Skia Arc — 270° sweep. Animated needle via Reanimated rotation. Center: score number counting up.	score: number (0-100), label?: string
RiskTemperatureGauge.tsx	Skia — segmented arc (10 steps Teal→Gold→Red). Pulsing outer ring at 3s interval keyed to risk level.	riskScore: number (0-100), riskLabel: string

1.4 AI Chat Interface — Technical

Aspect	Implementation
Streaming Transport	Server-Sent Events via Supabase Edge Function /ai-chat. Client reads via fetch() + ReadableStream — no WebSocket dependency.
Context Assembly	Before every request, claudeService.ts builds a structured context object: portfolio snapshot, last simulation result, user tier, recent chat history (last 8 turns).
Model Indicator	Edge Function response headers include X-QM-Model: haiku|sonnet|opus. chatStore reads this header and updates modelIndicator for display in the UI.
Optimistic Rendering	User message appended to chat immediately on send. Streaming assistant message rendered as it arrives. On error: last user message stays, error banner shown with retry.
Workflow Cards	Portfolio Doctor, Goal Check, What-If workflows return structured JSON alongside prose. claudeService.ts detects workflowType and renders a WorkflowCard component.
Rate Limiting (client)	UI disables send button during streaming. After response, enforces per-tier cooldown: Free 10 queries/day, Plus 30, Pro 100. Enforced server-side; client displays remaining count.
Offline Handling	TanStack Query persists last 20 messages to MMKV storage. Offline indicator shown; send disabled. On reconnect, queue is flushed.

.5 Authentication & Security

Feature	Implementation
Auth Provider	Supabase Auth — email+password, magic link, Apple Sign-In (iOS), Google Sign-In (Android via OAuth)
Token Storage	JWT + refresh token stored in expo-secure-store (iOS Keychain / Android Keystore). Never AsyncStorage.
Session Management	7-day JWT expiry, refresh token rotation enabled. Admin sessions: 8hr max age enforced client-side.
Biometric Auth	Optional Touch/Face ID re-authentication before viewing portfolio data. Uses expo-local-authentication. UI gate only — does not replace server-side JWT.
Deep Link Security	quantmind:// scheme registered in app.json. All deep-link params validated against allowlist before navigation. Unknown routes → home screen.
Jailbreak Detection	expo-device check on app start. Soft warning shown — access not blocked, but AI and simulation features log a risk_device_detected event.

1.6 Performance Targets & Optimization

Metric	Target	Implementation
Cold start	< 2.0 seconds	Hermes JS engine, deferred heavy imports (Skia, RevenueCat), splash screen until hydration complete
Navigation transition	< 300ms	Reanimated worklets on UI thread, no JS-thread bridge calls during transitions
Simulation result render	< 100ms	Skia on GPU thread, paths pre-computed before render pass, no layout calculation in draw loop
AI first token (TTFB)	< 1.2s	Edge Function cold start mitigated by keep-alive pings; Haiku routed for speed-critical paths
Chart interaction	60fps	All gesture handlers and animations on UI thread via Reanimated 3 worklets
Bundle size	< 12MB JS	Tree-shaking, dynamic imports for Pro-only features, SVG assets replaced with Skia paths
Memory ceiling	< 180MB	Simulation result arrays truncated to 200 sampled paths for display; full data on server only
Offline cache	80% feature parity	TanStack Query persistence to MMKV; portfolios, last simulation, AI history available offline

1.7 Testing Strategy

Test Type	Coverage
Unit Tests (Jest + RNTL)	All Zustand store actions, service layer methods, utility functions. Chart data transformation pure functions. Target: 80% coverage of src/services, src/store, src/utils.
Component Tests	React Native Testing Library — renders each screen with mocked stores, asserts key elements present, simulates user interactions. Covers auth flow, portfolio CRUD, simulation trigger.
E2E Tests (Maestro)	Device-level flows: Sign up → create portfolio → run simulation → view results → upgrade to Pro → run Pro features. Runs on EAS Device Farm on every PR.
Performance Tests	Flashlight (mobile perf profiler) — FPS measurement during chart interactions, simulation render, navigation. Fails CI if < 55fps sustained.

Test Type	Coverage
Visual Regression	Chromatic snapshots of all chart components via Storybook + react-native-storybook. Alerts on > 0.2% pixel diff.

02

Next.js 14 · quantmind.app · Vercel

The marketing website is the top-of-funnel conversion surface — a statically generated Next.js 14 site deployed to Vercel's edge network. Primary goals: App Store conversion, waitlist capture, and institutional/university partnership inquiries. SEO is a first-class concern — every route is server-rendered with full structured data.

2.1 Technology Stack

Layer	Technology
Framework	Next.js 14 App Router — SSG for all public routes, SSR for dynamic pricing/locale routes
Language	TypeScript 5.4 strict, ESLint + Prettier enforced in CI
Styling	Tailwind CSS v3 + FX1 design tokens via CSS custom properties. JIT mode, no unused styles in production.
Animation	Framer Motion — page transitions, scroll-triggered reveals, counter animations, interactive demo
3D / Canvas	Three.js r165 — particle field on hero, 3D portfolio universe teaser. react-three-fiber abstraction layer.
Forms	React Hook Form + Zod — waitlist form, contact form. Server Actions for form submission.
Email	Resend SDK — waitlist confirmation, partnership inquiry auto-reply, nurture sequence trigger
Analytics	Vercel Analytics (built-in) + custom analytics_events forwarded to Supabase via Edge Function
SEO	next-seo + Next.js metadata API. JSON-LD for Organization, Product, FAQ schemas. Sitemap generated by next-sitemap.
i18n	next-intl for future internationalisation. EN-US shipped at launch; structure supports EN-GB, FR, DE, JA expansion.
Testing	Playwright E2E, Jest + Testing Library for component tests, Lighthouse CI for performance gates
Deployment	Vercel — git-push deploys, preview URLs per PR, edge caching for static assets, ISR for dynamic content

2.2 Site Architecture & Route Map

```
apps/web/
├── app/
│   ├── page.tsx                    # Home: Hero + Features + Demo + Pricing + CTA
│   ├── layout.tsx                  # Root: fonts, analytics, cookie banner, metadata
│   ├── pricing/page.tsx            # Full pricing comparison, FAQ, trial CTA
│   ├── learn/
│   │   ├── page.tsx                # Learning hub — index of educational articles
│   │   ├── monte-carlo/page.tsx    # Deep-dive: how simulation works
│   │   ├── risk-metrics/page.tsx   # VaR, CVaR, Sharpe, Sortino explained
│   │   └── portfolio-theory/page.tsx
│   ├── blog/
│   │   ├── page.tsx                # Blog index — static MDX posts
│   │   └── [slug]/page.tsx         # Individual post — MDX rendered
│   ├── university/page.tsx         # Academic partnership landing page
│   ├── advisors/page.tsx           # Financial advisor landing page
```

```
│   ├── privacy/page.tsx          # GDPR/CCPA privacy policy
│   ├── terms/page.tsx            # Terms of service
│   ├── disclaimer/page.tsx       # Financial disclaimer
│   └── api/
│       ├── waitlist/route.ts     # POST — capture email, trigger Resend sequence
│       └── contact/route.ts      # POST — partnership/advisor inquiry
├── components/
│   ├── sections/                 # HeroSection, FeaturesSection, DemoSection...
│   ├── ui/                       # Button, Badge, Card, Input, Modal primitives
│   ├── demo/                     # InteractiveMonteCarlo — real simulation in-browser
│   └── layout/                   # Navbar, Footer, CookieBanner, AnnouncementBar
├── content/blog/                 # MDX blog posts with frontmatter
└── public/                       # og-image.png, sitemap.xml, robots.txt, icons
```

The hero demo section embeds a real Monte Carlo simulation running client-side via WebAssembly. The Python simulation engine is compiled to WASM via Pyodide, with a lightweight JS wrapper. Users drag sliders and see 500-path fan charts update in < 100ms — converting visitors far better than a static screenshot.

Feature	Implementation
Engine	Python simulation core compiled to WASM via Pyodide 0.25. Loaded lazily — does not block page load.
Initial Load	WASM binary streamed and compiled in background after above-fold content is interactive. Progress indicator shown.
Interactivity	Volatility (0–80%), Expected Return (−10% to +30%), Time Horizon (1–10Y), Number of Assets (1–5). All sliders mapped to simulation parameters.
Rendering	Chart.js 4 with custom neon plugin — Cyan median line, gradient bands. Re-renders on every slider change with 200ms debounce.
Fallback	If WASM fails to load or browser unsupported: pre-computed sample results displayed with a 'Full simulation requires the app' CTA.
Conversion CTA	After user interacts with demo for > 15 seconds: a slide-up banner appears — 'Run 10,000 paths with your actual portfolio → Download App'. Dismissed once per session.

4 SEO & Performance Architecture

Feature	Implementation
Core Web Vitals Targets	LCP < 1.8s, FID/INP < 100ms, CLS < 0.05. Enforced via Lighthouse CI — fails PR if below thresholds.
Image Optimization	Next.js Image component throughout — WebP/AVIF, lazy loading, blur-up placeholder, responsive sizes. No raw <img> tags.
Font Strategy	Space Grotesk + JetBrains Mono loaded via next/font — self-hosted, preloaded, font-display:swap.
Critical CSS	Tailwind purged to < 15KB. Above-fold CSS inlined. Below-fold CSS loaded asynchronously.
Third-Party Scripts	All analytics deferred until cookie consent given. No tag manager — direct script tags only. Google Analytics via Partytown worker.
Structured Data	Organization, Product, FAQPage, Article, BreadcrumbList JSON-LD injected per route via next-seo.
Sitemap	Auto-generated by next-sitemap, including blog posts, learn articles, all static routes. Submitted to Google Search Console.

Feature	Implementation
Open Graph	Unique og:image per route via Vercel OG (@vercel/og) — dynamically generated images with Quantmind branding for each article/page.

2.5 Waitlist & Email Funnel

Stage	Implementation
Capture	Email + first name form on hero and footer. Server Action validates with Zod, writes to waitlist table in Supabase, triggers Resend sequence.
Welcome Email	Sent immediately — includes app download links, 3 core use cases, link to interactive demo.
Day 3 Nurture	'You won't believe what a 28% probability of loss looks like' — hooks user into Monte Carlo concept, links to learn/monte-carlo article.
Day 7 Nurture	'Compare your portfolio risk to 1,000 others' — social proof angle, trial CTA with countdown.
Day 14 Re-engagement	'Your free trial is waiting' — final reminder before waitlist entry expires.
Referral	Share link appended to all emails — referral tracked via utm_source=waitlist_refer. Referred signups flagged in Supabase for future rewards program.

Supabase · FastAPI · Deno Edge Functions · PostgreSQL

The Quantmind backend is a hybrid architecture: Supabase provides the primary database, authentication, realtime subscriptions, and serverless Edge Functions (Deno runtime); a standalone Python FastAPI microservice handles the computationally intensive Monte Carlo simulation engine; and RevenueCat manages subscription lifecycle events.

3.1.1 PostgreSQL Schema

```
-- Core tables
users               -- id, email, created_at, app_metadata (role, tier)
portfolios          -- id, user_id, name, assets[], correlation_matrix, created_at
assets              -- id, portfolio_id, ticker, weight, expected_return, volatility
simulations         -- id, user_id, portfolio_id, params_hash, result JSONB, created_at
simulation_paths    -- id, simulation_id, paths BYTEA (compressed Float64Array)

-- Subscription & billing
subscriptions       -- id, user_id, tier, status, trial_end, revenuecat_customer_id
webhook_events      -- id, provider, event_id (unique), payload JSONB, processed_at

-- AI & analytics
analytics_events    -- id, user_id, event_type, props JSONB, created_at
ai_sessions         -- id, user_id, model_id, task_type, tokens_in, tokens_out, latency_ms
prompt_templates    -- id, name, version, content, model_target, status, published_at

-- Operations
feature_flags       -- id, key, rules JSONB, enabled, updated_at, updated_by
flag_audit_log      -- id, flag_id, actor_id, change JSONB, reason, created_at
ota_releases        -- id, channel, bundle_hash, rollout_pct, status, created_at
support_tickets     -- id, user_id, subject, status, priority, created_at
support_messages    -- id, ticket_id, author_id, body, is_ai_draft, created_at
admin_audit_log     -- id, actor_id, action, target_type, target_id, meta JSONB, ts
```

3.1.2 Row-Level Security Policies

Table	Policy
portfolios	USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id) — users isolated to own rows
simulations	SELECT: auth.uid() = user_id. INSERT: auth.uid() = user_id. No UPDATE/DELETE from client — managed via Edge Function.
analytics_events	INSERT only from client: WITH CHECK (auth.uid() = user_id). No SELECT policy for users — admin reads via service role only.
support_tickets	Users: SELECT/INSERT own tickets. No UPDATE from client. Admin reads all via service role.
feature_flags	No client access — read via Edge Function that hydrates flags per user session. Write: service role (dashboard) only.
prompt_templates	No client access. Published templates read by ai-chat Edge Function. CRUD: service role (dashboard) only.
admin_audit_log	INSERT only via service role Edge Functions. No UPDATE/DELETE policy exists — log is immutable by design.

3.1.3 Edge Functions — Complete Inventory

Function	Route	Responsibility
health	GET /health	Liveness probe — returns {status, version, timestamp}. Used by uptime monitors and dashboard health indicator.
portfolios	GET/POST/PUT/DELETE /portfolios	Full CRUD for portfolios table. Auth + RLS enforced. Validates weights sum to 1.0, tickers format. Returns typed PortfolioResponse.
assets	GET /assets/search?q=	Proxies ticker search to Yahoo Finance API. Caches results 1hr in Supabase. Returns {ticker, name, exchange, assetClass}.
assets-history	GET /assets/history	Fetches historical OHLCV from Yahoo Finance. Used for auto-populated expected return and volatility suggestions.
simulate	POST /simulate	Validates SimulationRequest, checks tier entitlement (path limit), writes job to simulation queue, returns {jobId, estimatedMs}.
simulate-status	GET /simulate-status/:id	Polls simulation microservice for job status. Streams partial results when available. Returns SimulationResult on completion.
ai-chat	POST /ai-chat	Multi-model AI proxy. Assembles context, classifies task, routes to correct model, streams response. Logs to ai_sessions.
ai-task	POST /ai-task	Batch AI tasks (Portfolio Doctor, stress test debrief, PDF narrative). Non-streaming. Returns structured WorkflowResult.
webhook-revenuecat	POST /webhook/revenuecat	Verifies HMAC signature, processes subscription events, updates subscriptions table. Idempotent via event_id unique constraint.
feature-flags	GET /feature-flags	Returns active flags for the authenticated user, evaluated against their tier, cohort, and platform. Cached per user 5 min.
support-tickets	GET/POST/PUT /support	Create ticket, list user's tickets, add message, update status. Admin endpoints for bulk management.
analytics-ingest	POST /analytics	Batch event ingestion from mobile and web. Validates schema, writes to analytics_events. Rate-limited by user.
pdf-report	POST /pdf-report	Generates simulation PDF report. Calls AI for narrative sections. Uses Puppeteer-core via Deno for PDF rendering. Uploads to Supabase Storage.

3.2.1 Architecture

The simulation engine is a standalone Python FastAPI service deployed as a Docker container. It is invoked exclusively by the Supabase simulate Edge Function — not directly accessible from clients. Authentication uses a shared secret (SIMULATION_SECRET_KEY) validated on every request.

Layer	Technology
Runtime	Python 3.12 — CPython for maximum numpy/scipy performance
Framework	FastAPI 0.111 + Pydantic v2 — async request handling, typed models, OpenAPI docs auto-generated
Numerics	NumPy 2.0, SciPy 1.13 — vectorised matrix operations, distribution sampling, statistical functions

Layer	Technology
Concurrency	asyncio + ProcessPoolExecutor — simulation runs in worker processes (not threads) to bypass GIL. Up to 4 workers per container.
Containerisation	Docker — python:3.12-slim base, multi-stage build (builder + runtime). Image < 280MB.
Deployment	Railway.app — auto-scaling, health check endpoint, environment variable management, log streaming
Monitoring	Structured JSON logging (structlog) → Railway logs → forwarded to Sentry for error alerting
Job Queue	Redis (Upstash) — simulation jobs enqueued by Edge Function, dequeued by FastAPI worker. Enables horizontal scaling without duplicate processing.

3.2.2 Engine Module Map

```
apps/simulation/
├── app/
│   ├── main.py              # FastAPI app factory, CORS, lifespan, startup checks
│   ├── config.py            # Pydantic Settings — env vars, tier limits, model params
│   ├── dependencies.py      # Auth middleware, rate limiting, tier enforcement
│   ├── routers/
│   │   ├── simulate.py      # POST /simulate — validates, dispatches to engine
│   │   ├── scenarios.py     # POST /scenarios — built-in + custom stress scenarios
│   │   ├── goals.py         # POST /goals — retirement / education goal probability
│   │   ├── optimize.py      # POST /optimize — allocation optimization engine
│   │   └── health.py        # GET /health — liveness + readiness
│   ├── models/
│   │   ├── portfolio.py     # PortfolioInput, AssetInput, CorrelationMatrix (Pydantic)
│   │   ├── simulation.py    # SimulationParams, SimulationResult, PercentilePaths
│   │   ├── metrics.py       # RiskMetrics, GoalResult, OptimizationResult
│   │   └── scenario.py      # ShockParams, BuiltInScenario enum
│   └── engine/
│       ├── gbm.py           # GBM path generator — vectorised NumPy
│       ├── cholesky.py      # Correlated random normals via Cholesky decomposition
│       ├── monte_carlo.py   # Orchestrator — multi-asset, multi-path simulation
│       ├── risk_metrics.py  # VaR, CVaR, Sharpe, Sortino, max drawdown
│       ├── diversification.py # HHI score, avg correlation, composite 0-100
│       ├── scenarios.py     # apply_shock() — 2008/COVID/crypto/custom
│       ├── goals.py         # Goal probability — Monte Carlo retirement/education
│       ├── optimizer.py     # Risk-adjusted allocation improvement (gradient-based)
│       └── advanced/
│           ├── jump_diffusion.py   # Merton jump-diffusion
│           ├── fat_tails.py        # Student-t, stable distribution returns
│           └── regime_switching.py # Markov bull/bear regime model
└── tests/
    ├── test_gbm.py          # E[S_T/S_0] ≈ exp(μT), normality, BSM option pricing
    ├── test_cholesky.py     # Realised vs target correlation ±0.02
    ├── test_risk_metrics.py # 15+ unit tests, textbook reference values
    └── test_api.py          # FastAPI integration tests with httpx AsyncClient
```

3.2.3 Core Algorithm Detail

Algorithm	Implementation
GBM Path Generation	dS = μS·dt + σS·dW. Vectorised NumPy: paths matrix shape (n_paths × n_steps × n_assets). dt = 1/252 (daily). Random normals generated once, reused across correlation transforms.

Algorithm	Implementation
Cholesky Correlation	L = cholesky(Σ). Correlated_Z = Z_independent @ L.T. Validated pre-decomposition: matrix symmetry check, eigenvalue check (all λ > -1e-10 tolerance).
Simulation Batching	10,000-path simulation split into 10 batches of 1,000. Each batch computed, risk metrics updated, partial result streamed to Edge Function. Enables progressive fan chart rendering.
Risk Metric Computation	VaR: np.percentile(terminal_values, 5). CVaR: mean of values < VaR. Sharpe: (μ_portfolio - r_f) / σ_portfolio. Max drawdown: vectorised peak-to-trough across all paths.
Jump Diffusion (Pro)	Merton model: J ~ Poisson(λ·dt). Jump size ~ LogNormal(μ_J, σ_J). Applied as multiplicative shock to GBM path at each jump event.
Regime Switching (Pro)	Two-state Markov chain (bull/bear). Transition matrix P calibrated to historical regime durations. Volatility multiplied by regime_multiplier per state per time step.
Goal Probability	P(goal_achieved) = count(paths where terminal_value ≥ goal_target) / n_paths. Confidence interval computed via bootstrap on path sample.
Portfolio Optimization	Gradient-based allocation optimizer using scipy.optimize.minimize with SLSQP. Objective: maximize Sharpe ratio subject to weights ∈ [0,1], sum=1. Returns 5 candidate rebalancing suggestions.

3.3.1 Task Router — packages/ai

Function	Implementation
classifyTask(input)	Lightweight Haiku call: system prompt maps input to TaskType enum in < 80ms. Falls back to rule-based keyword matching if confidence < 0.85. Returns: {taskType, urgency, requiredTier, estimatedTokens}.
routeToModel(task, tier)	Maps {taskType × tier} to {modelId, executionMode, streaming}. Tier gate enforced: Opus calls blocked if tier < Pro. Returns ModelConfig object.
executeTask(task, model)	Dispatches to Anthropic API with assembled messages[], tools[], system prompt. Streaming: returns ReadableStream. Batch: awaits full response. Retries: exponential backoff (1s, 2s, 4s), circuit-breaker on 3 consecutive 529s.
aggregateResponses(subTasks[])	Runs up to 3 sub-tasks in parallel via Promise.all where context windows are independent. Coherence check: detects contradictions in numerical outputs. Returns merged WorkflowResult.
Prompt Versioning	Each prompt template identified by {name, version}. Active version resolved at runtime from prompt_templates table. Rollback: set previous version to status='active', current to 'deprecated' — takes effect immediately without redeploy.
Context Assembly	context.ts builds: portfolioSnapshot (tickers, weights, last simulation summary), userTier, conversationHistory (last 8 turns), currentDate. PII stripped (no email, name, account ID) before assembly.

3.3.2 Model Routing Strategy

Task Category	Model	Rationale	Tier Access
Real-time metric explanation	Claude Haiku	Low latency, cost-efficient for short responses	Free+

Task Category	Model	Rationale	Tier Access
Portfolio weakness analysis	Claude Sonnet	Balanced reasoning depth & speed	Plus+
Goal-based planning narrative	Claude Sonnet	Multi-step reasoning, structured output	Plus+
Portfolio Doctor deep-dive	Claude Opus	Maximum analytical depth & nuance	Pro
Stress test interpretation	Claude Opus	Complex scenario reasoning	Pro
Keyword / intent classification	Haiku (fine-tuned)	Microsecond routing decisions	All
PDF report narrative generation	Sonnet	Long-form structured writing	Pro
Chat auto-reply drafts (support)	Haiku	Fast first-pass support drafts	Admin

3.3.3 AI Analytics Instrumentation

Feature	Implementation
Event Schema	Every model call logged to ai_sessions: {id, user_id, session_id, model_id, task_type, input_tokens, output_tokens, latency_ms, ttfb_ms, outcome_feedback, workflow_id, feature_flag_overrides, created_at}
Cost Attribution	output_tokens × model_price_per_1k_tokens aggregated per user, per feature, per tier. Surfaced in dashboard AI analytics tab with daily/weekly views.
Latency Tracking	ttfb_ms: time to first byte (streaming start). latency_ms: full response completion. P95 alert if ttfb > 2s for Haiku, > 5s for Sonnet, > 10s for Opus.
Quality Scoring	outcome_feedback: -1 (thumbs down), 0 (no feedback), 1 (thumbs up). Rolling 7-day quality score per prompt_template version. Threshold: < 0.6 triggers review alert in dashboard.
Token Budget	Daily token budget per tier: Free 10K, Plus 50K, Pro 200K. Soft limit: warning banner in app at 80%. Hard limit: 429 response with Retry-After header.

3.4.1 Environment Configuration

Variable	Surface	Secret?	Rotation
SUPABASE_SERVICE_ROLE_KEY	Admin/Dashboard server-side	Yes	On team change
ANTHROPIC_API_KEY	Edge Functions (server only)	Yes	Monthly
REVENUECAT_WEBHOOK_SECRET	webhook-revenuecat fn	Yes	On compromise
SIMULATION_SECRET_KEY	Edge Fns + FastAPI	Yes	Every deploy
EAS_ACCESS_TOKEN	GitHub Actions CI	Yes	Quarterly
UPSTASH_REDIS_URL / TOKEN	Edge Fns + FastAPI	Yes	Quarterly
EXPO_PUBLIC_SUPABASE_ANON_KEY	Mobile (safe to expose)	No	Never
SENTRY_DSN	All apps	No	N/A
RESEND_API_KEY	Web server-side	Yes	Quarterly

3.4.2 CI/CD Pipeline

Workflow	Description
ci.yml	On every push: TypeScript compile (tsc --noEmit), ESLint, Prettier check, Jest unit tests. Blocks merge if any step fails.
python-ci.yml	On simulation/ changes: ruff lint, black format check, pytest with coverage report. Blocks if coverage < 80% on engine/ modules.
mobile-preview.yml	On PR: EAS Build (preview profile) → QR code comment on PR for device testing. Runs Maestro smoke tests on emulator.
deploy-web.yml	On main merge: Vercel production deploy. Lighthouse CI gate (LCP < 1.8s, CLS < 0.05). Visual regression check.
deploy-dashboard.yml	On main merge: Vercel production deploy (private domain). Smoke test: health check endpoint responds 200.
deploy-simulation.yml	On simulation/ tag push: Docker build + push to GitHub Container Registry. Railway auto-deploys from GHCR on new tag.
release-mobile.yml	Manual trigger: EAS production build + EAS Submit to App Store Connect + Google Play Internal Track. Generates changelog from merged PR titles.
security.yml	Weekly: TruffleHog secret scan, npm audit --audit-level=high, pip-audit. Alerts via Slack on findings.

3.4.3 Database Migration Strategy

Practice	Implementation
Versioning	Sequential numbered files: NNN_description.sql. Applied in order via supabase db push. Never edit a committed migration — always write a new one.
Forward-Only	No down migrations in production. If a migration must be reversed, write a corrective migration NNN_revert_description.sql.
Zero-Downtime	Additive changes (new columns with defaults, new tables) deployed with no downtime. Destructive changes: multi-step — add new column → backfill → remove old column across separate deploys.
Seed Data	supabase/seed/ contains development seed data: demo user, sample portfolios, sample simulation results. Never run in production.
Backup	Supabase daily automated backups retained 7 days (paid plan). Point-in-time recovery available. Weekly manual pg_dump to S3 for additional retention.

Next.js 14 · admin.quantmind.app · Internal Operations

The management dashboard is a single internal application that provides full operational control over the Quantmind platform: user management, AI configuration, OTA update orchestration, feature flag governance, analytics, and support operations. It is built with Next.js 14 App Router, secured via email allowlist authentication, and uses the Supabase service role key exclusively server-side.

Layer	Technology
Framework	Next.js 14 App Router — Server Components for data-heavy views, Client Components for interactive controls
Auth	NextAuth v5 — email allowlist + magic link. Admin role validated from Supabase app_metadata on every server request.
Database	Supabase service role client (server-side only). Never in browser — all DB access via Server Actions or Route Handlers.
Charts	Recharts 2.12 + custom FX1-themed wrappers (dark backgrounds, Cyan primary series, neon glow on hover)
State	TanStack Query v5 — background refresh every 30s for live KPI cards. Optimistic updates for flag toggles and tier changes.
Forms	React Hook Form + Zod — all admin action forms validated client + server side.
Tables	TanStack Table v8 — sortable, filterable, paginated. Virtual rows for large datasets (users list > 10,000 rows).
OTA	EAS Update REST API — bundle publish, rollout control, device coverage, emergency revert.
Monitoring	Sentry (error tracking) + Vercel Analytics (performance) + custom health polling for all service endpoints.
Deployment	Vercel — private domain (admin.quantmind.app), IP allowlist optional, forced HTTPS, HTTP security headers.

4.2 Dashboard Architecture

```
apps/dashboard/
├── app/
│   ├── layout.tsx              # Auth guard, sidebar, top bar, notification context
│   ├── (auth)/login/page.tsx   # Magic link login — email allowlist enforced
│   ├── overview/page.tsx       # Real-time KPI cards, system health, active alerts
│   ├── app-management/
│   │   ├── page.tsx            # OTA releases list, publish button, channel selector
│   │   ├── ota/page.tsx        # Bundle history, rollout control, device coverage chart
│   │   ├── feature-flags/
│   │   │   ├── page.tsx        # Flag list — search, filter, toggle, audit trail
│   │   │   └── [id]/page.tsx   # Flag detail — targeting rules editor, experiment config
│   │   └── releases/page.tsx   # EAS build board, App Store submission, changelog editor
│   ├── ai-control/
│   │   ├── page.tsx            # Model routing overview, cost summary
│   │   ├── prompts/
│   │   │   ├── page.tsx        # Template library — list, version history
│   │   │   └── [id]/page.tsx   # Template editor — Draft/Review/Publish workflow
│   │   ├── playground/page.tsx # Live prompt tester — model selector, context simulator
│   │   └── analytics/page.tsx  # Model usage, cost attribution, latency, quality charts
```

```
│   ├── users/
│   │   ├── page.tsx            # User table — search, filter by tier/status/date
│   │   └── [id]/page.tsx       # User detail — profile, portfolios, simulations, actions
│   ├── analytics/
│   │   ├── page.tsx            # Growth overview — signups, MRR, DAU
│   │   ├── revenue/page.tsx    # MRR/ARR breakdown, cohort LTV, churn
│   │   ├── engagement/page.tsx # DAU/WAU/MAU, feature usage heatmap, retention curves
│   │   └── simulations/page.tsx # Simulation volume, tier distribution, error rates
│   ├── support/
│   │   ├── page.tsx            # Ticket queue — SLA countdown, priority, assignment
│   │   └── [id]/page.tsx       # Ticket thread — user context, AI draft, send/escalate
│   └── api/
│       ├── users/[id]/route.ts       # Tier change, suspend, delete, impersonate
│       ├── feature-flags/route.ts    # CRUD + audit log
│       ├── ota/route.ts              # EAS Update API proxy
│       └── ai-config/route.ts        # Prompt template CRUD, model routing config
├── components/
│   ├── charts/    # ModelUsageChart, RevenueChart, RetentionHeatmap...
│   ├── tables/    # UserTable, TicketTable, FlagTable (TanStack Table)
│   ├── forms/     # FlagEditor, PromptEditor, UserActionForm
│   ├── layout/    # Sidebar, TopBar, NotificationToast, BreadcrumbNav
│   └── shared/    # KPICard, StatusBadge, AuditTrail, HealthIndicator
└── lib/
    ├── supabaseAdmin.ts  # Service-role client — server-side only
    ├── auth.ts           # NextAuth config, requireAdmin(), enforceAdminAge()
    ├── easClient.ts      # EAS Update REST API wrappers
    └── analytics.ts      # SQL aggregation helpers for dashboard queries
```

4.3.1 Overview Module

Component	Details
System Health Grid	Live polling (30s) of all service endpoints: Supabase DB, simulation FastAPI, Anthropic API reachability, RevenueCat webhook last received. Red/Amber/Green status indicators.
KPI Cards (real-time)	Total Users, MRR, DAU, Active Trials, Churn Rate, Crash Rate. Each card has a 24hr sparkline. Values count up on load. Flash animation on live refresh.
Active Alerts	Incidents feed — pulled from admin_audit_log WHERE event_type IN ('crash_rate_spike','simulation_timeout','ai_budget_alert','ota_paused'). Dismissible with reason log.
Revenue Pulse	Mini chart: MRR for last 30 days. Annotation markers on EAS release dates to correlate releases with revenue events.

4.3.2 App Management Module

Feature	Details
OTA Release Board	Lists all EAS Update publications. Columns: version, channel, rollout %, device adoption %, crash rate, status. Inline actions: Pause Rollout, Advance to Next Stage, Emergency Revert.
Staged Rollout Logic	5% → 25% → 50% → 100%. Auto-advance after 2hr at each stage if crash rate < threshold. Auto-pause if crash rate > threshold within 15min of publish. Both thresholds configurable per channel.
Feature Flag Editor	Inline targeting rule builder — dropdowns for dimension (tier/cohort/platform/version/percentage), operator (is/is_not/includes/gt/lt), value. Real-time rule preview.

Feature	Details
A/B Experiment Manager	Model experiments: select task_type, traffic split %, model A vs model B, success metric (quality_score/latency_p95/cost_per_call), end date. Live progress chart of metrics as experiment runs.
Changelog Editor	WYSIWYG (TipTap) editor for release notes. Auto-populated from merged PR titles via GitHub Releases API. Draft → Published workflow.
Crash Rate Gate	Per-release threshold configuration. If crash rate exceeds threshold: rollout auto-paused, Slack webhook fired, dashboard alert created, email sent to platform team DL.

4.3.3 AI Control Centre

Feature	Details
Model Routing Config	Table view of all task_type → model mappings. Inline edit — dropdown to select Haiku/Sonnet/Opus per task. Override column: select feature flag to A/B test routing. Changes take effect on next Edge Function cold start (< 30s).
Prompt Playground	Three-panel layout: (1) Template selector + parameter inputs, (2) Simulated portfolio context builder, (3) Live model response with token count, cost estimate, latency meter. Side-by-side mode: run same prompt on two models simultaneously.
Prompt Versioning	Template list shows: name, current version, status (Draft/Review/Published/Deprecated), quality score, publish date. Version history drawer: diff between versions. Publish button: requires reviewer sign-off comment, creates immutable snapshot.
AI Analytics	7 chart views: Model Usage Distribution, Cost Attribution, Latency Percentiles (P50/P95/P99), Task Classification Accuracy, Quality Scores (rolling 7-day per template), Workflow Completion Rate, Token Budget arc gauge.

4.3.4 User Management Module

Feature	Details
User Table	TanStack Table — 100 rows/page, virtualized. Columns: avatar, email, tier badge, signup date, last active, sim count, AI query count. Filters: tier, status (active/suspended/deleted), signup date range, search by email.
User Detail Page	Left panel: account info, subscription history timeline, avatar. Right panel: portfolios list (expandable), last 5 simulations, last 5 AI sessions, support ticket count. Bottom: action buttons.
Tier Controls	Dropdown to upgrade/downgrade tier. Change logged to admin_audit_log with actor_id, reason (required), and timestamp. RevenueCat tier not modified — for support-only overrides in Supabase only.
Impersonation Mode	Generates a short-lived (15min) impersonation token for the target user. Redirects to mobile-equivalent web view. All actions logged. Cannot modify data — read-only enforcement at service role level.
Trial Management	Extend trial: updates trial_end in subscriptions table. Revoke trial: sets trial_end to now(). Both logged. Cohort view: trial starts by day, conversion funnel (started→active user→paid).

4.3.5 Analytics Module

View	Charts	Key Metrics
Growth Overview	Signups area chart (7D/30D/90D toggle), DAU bar chart, WAU/MAU line overlay	New signups/day, D1/D7/D30 retention, DAU/MAU ratio

View	Charts	Key Metrics
Revenue	MRR waterfall (new MRR + expansion − contraction − churn), ARR projection, tier decomposition pie	MRR, ARR, Net Revenue Retention, Churn Rate, ARPU
Cohort Analysis	Cohort retention heatmap (signup week × weeks retained). Color: Red→Teal scale.	W1/W4/W12 retention by cohort, tier upgrade lag
Engagement	Feature usage heatmap (feature × day of week). Simulations/user/month histogram.	Simulations/MAU, AI queries/MAU, PDF exports/Pro user
Conversion Funnel	Multi-step funnel: Install → Sign Up → First Sim → Trial Start → Paid → Pro Upgrade	Each stage conversion %, drop-off identification
Simulation Analytics	Volume by tier (stacked bar), failure rate sparkline, avg path count, median duration trend	Simulations/day, P95 duration, error rate %

4.3.6 Support Module

Feature	Details
Ticket Queue	Table: subject, user (tier badge), priority, SLA countdown (colour: Green→Amber→Red as deadline approaches), assignee, status. Filters: open/pending/resolved, priority, unassigned. Bulk actions: assign, close, escalate.
AI Auto-Draft	On ticket open: Haiku queries knowledge_base for relevant FAQs, drafts first-pass reply. Draft shown in editor — admin clicks Accept/Edit/Discard. Accepted drafts: quality_score incremented, sent to user.
Escalation Workflow	High-priority trigger conditions: user tier=Pro + unresolved > 4hr, crash report attached, payment dispute keyword detected. Auto-action: assign to senior queue, Slack webhook to #support-escalations.
Knowledge Base Editor	TipTap WYSIWYG editor for FAQ articles. Each article tagged with topics (simulation/ai/billing/auth). Published articles injected into AI auto-draft context window. Version history tracked.
SLA Configuration	Default SLAs: Pro tier 12hr first response, Plus 24hr, Free 72hr. Configurable per ticket priority. SLA breach: ticket turns Red in queue, automatic escalation trigger fired.

Security Layer	Implementation
Admin Role Enforcement	requireAdmin() called at top of every Server Action and Route Handler. Reads auth.uid() → fetches user.app_metadata.role from Supabase. Returns 403 if role !== 'admin'. app_metadata is service-role-only writable.
Session Age Enforcement	Admin JWTs: 8hr max age. enforceAdminAge() called on every page load — redirects to login if session older than 8hr. Separate from standard user 7-day JWT.
MFA Requirement	Admin users without MFA enrolled are redirected to MFA setup screen on every login attempt until enrolled. TOTP via Supabase Auth MFA. 8 single-use backup codes stored hashed.
Audit Immutability	admin_audit_log table: no UPDATE/DELETE policy exists at RLS level. Service role only allowed INSERT. Ensures complete tamper-proof record of all admin actions.

Security Layer	Implementation
Impersonation Safeguards	is_impersonation: true header on all impersonation requests. Server-side: write operations return 403 when this header present. Impersonation sessions expire after 15 minutes regardless of activity.
Secret Isolation	SUPABASE_SERVICE_ROLE_KEY, ANTHROPIC_API_KEY, EAS_ACCESS_TOKEN present only in Vercel environment for dashboard deployment. Never in mobile app, never in any client bundle.

05

Shared concerns across all four pillars

5.1 Monorepo Structure

```
quantmind/ (private GitHub — npm workspaces + Turborepo)
├── apps/
│   ├── mobile/       # React Native / Expo
│   ├── web/          # Next.js marketing site
│   ├── dashboard/    # Next.js management dashboard
│   └── simulation/   # Python FastAPI
├── packages/
│   ├── ai/           # Multi-model task router, workflows, prompts
│   ├── shared-types/ # TypeScript interfaces shared across all apps
│   ├── ui/           # React Native design system (FX1 tokens)
│   └── analytics/    # Typed event catalog, tracking utilities
├── supabase/
│   ├── migrations/   # Versioned SQL — 001 through 012
│   ├── functions/    # Edge Functions (Deno)
│   └── seed/         # Development seed data
├── .github/workflows/ # CI/CD pipelines
├── turbo.json         # Turborepo pipeline (build · test · lint)
└── package.json       # Root workspace config
```

@quantmind/shared-types provides TypeScript interfaces shared across all TypeScript applications. Generated from the Supabase schema using supabase gen types typescript — the database.types.ts file is the single source of truth for all table shapes.

Domain	Types
Portfolio	Portfolio, PortfolioInput, Asset, AssetInput, CorrelationMatrix — used by mobile, web, dashboard, and Edge Functions
Simulation	SimulationParams, SimulationRequest, SimulationResult, PercentilePaths, RiskMetrics — mobile to FastAPI contract
User	User, SubscriptionTier (Free|Plus|Pro|Student), Entitlement, SessionInfo — auth + subscription across all apps
AI	ChatMessage, ChatRole, TaskType, ModelId, WorkflowResult, WorkflowCard — mobile AI interface + Edge Function contract
Events	AnalyticsEvent, EventName (union type of all event names), EventProps — typed event catalog for analytics service
Dashboard	FeatureFlag, FlagRule, OTARelease, PromptTemplate, SupportTicket — dashboard-specific types

5.3 Observability Stack

Layer	Tool	Coverage
Error Tracking	Sentry — all 4 surfaces. scrubFields: password, token, api_key, portfolio, assets. Source maps uploaded on every deploy.	Crash reports, unhandled promise rejections, performance traces
Uptime Monitoring	Better Uptime — HTTP checks every 60s on: Supabase health,	Incident detection, SLA calculation, historical uptime report

Layer	Tool	Coverage
	FastAPI /health, dashboard health, web Lighthouse score.	
Log Aggregation	Railway structured logs (FastAPI) + Vercel logs (Next.js apps) + Supabase logs. All surfaced in dashboard via embedded log viewer.	Simulation errors, Edge Function failures, DB query performance
Performance	Vercel Analytics (web + dashboard), Flashlight (mobile). Lighthouse CI gates on web deploy. RN Perf Monitor in dev.	Core Web Vitals, mobile FPS, API response percentiles
AI Observability	ai_sessions table + dashboard AI analytics tab. Custom — no third-party LLM monitoring tool to avoid sending prompt data externally.	Token costs, latency, quality scores, model routing accuracy
Database	Supabase Studio + pg_stat_statements. Slow query log (> 1s). Index usage monitoring.	Query performance, connection pool utilization, RLS evaluation cost

5.4 Subscription Tier Matrix

Feature	Free	Plus ($9.99/mo)	Pro ($24.99/mo)
Simulation paths	2,000	10,000	10,000
Portfolios	1–2	Unlimited	Unlimited
Advanced models (jump/fat-tail/regime)	No	No	Yes
Stress testing (custom scenarios)	No	Basic presets	Full + custom builder
Goal-based planning	No	No	Yes
Diversification Score	No	Yes	Yes
Risk Temperature Gauge	No	Yes	Yes
AI model access	Haiku only	Haiku + Sonnet	Haiku + Sonnet + Opus
AI queries / day	10	30	100
PDF report export	No	No	Yes
Smart risk alerts	No	No	Yes
Portfolio Doctor	No	No	Yes (Opus)
Correlation network (3D)	No	No	Yes
API access (future)	No	No	Yes

5.5 16-Week Delivery Roadmap

Phase	Weeks	Focus	Key Deliverables
1 — Foundation	1–3	Auth, portfolio CRUD, navigation, DB schema	Supabase migrations 001–003, mobile auth + portfolio screens, Edge Functions health/portfolios/assets

Phase	Weeks	Focus	Key Deliverables
2 — Simulation	4–6	Monte Carlo engine, GBM, risk metrics, charts	FastAPI simulation service + Docker, fan chart (Skia), risk metrics grid, simulation polling client
3 — AI Layer	7–9	Multi-model router, streaming, workflows, context assembly	packages/ai/ complete, mobile chat screen, Portfolio Doctor, ai-chat + ai-task Edge Functions
4 — Premium + Dashboard	10–12	Subscriptions, scenarios, OTA, feature flags, dashboard v1	RevenueCat integration, stress test module, dashboard overview + app-management + ai-control modules
5 — Polish	13–14	Performance, accessibility, onboarding, FX1 visual effects	RN Skia animations, skeleton loaders, Maestro E2E, dashboard analytics + support modules
6 — Launch	15–16	App Store submission, web site, public repo, press kit	EAS production build + submit, quantmind.app live, quantmind-showcase/ complete, Lighthouse CI green

6.1 Tiered Subscription Pricing

Tier	Price	Emotional Value Delivered
Free — Explorer	$0	"Wow, this is eye-opening."
Plus — Investor	$9.99/month or $99/year	"This helps me invest smarter."
Pro — Quantmind Pro	$24.99/month or $229/year	"This is a serious financial tool."
Student / Academic	$5/month (verified)	"Powerful learning at student price."

Annual subscriptions save 17–23%. A 7-day free trial (no card required) is available for all paid tiers, with a data-driven nurture sequence to drive trial-to-paid conversion.

Dimension	Goal
Active Users	500,000+ active users
Paying Subscribers	25,000+ paying subscribers
ARR	$5M+ Annual Recurring Revenue
Institutional	Partnerships with 100+ universities
Quality	4.8+ App Store rating
Market Position	Recognised as the de facto mobile platform for forward-looking portfolio risk analysis

- Fully autonomous portfolio optimisation with predictive AI risk alerts
- Multi-client management and white-label solutions for financial advisors
- International markets, alternative asset classes, options and derivatives modelling
- ESG and impact investing metrics
- Institutional enterprise security features and custom model development

ASTERIX is an aviation surveillance data protocol engineered for high-frequency, multi-entity real-time tracking. While Quantmind does not integrate ASTERIX directly, its architectural principles map precisely onto the platform's live simulation and risk monitoring needs. The concept translation is straightforward: aircraft positions become asset prices; radar update ticks become price ticks; simultaneous aircraft tracks become parallel Monte Carlo simulation paths.

7.1.1 Real-Time Streaming Pipeline

ASTERIX systems are purpose-built for high-frequency incoming data with continuous positional updates. Quantmind applies this same pattern to live portfolio valuation: asset prices stream in via WebSockets (Twelve Data) and Supabase Realtime subscriptions, triggering incremental VaR recalculation as the market moves. The simulation engine processes these updates in batches — mirroring how ASTERIX radar sweeps arrive in discrete, ordered bursts rather than as raw continuous signals.

7.1.2 Multi-Entity Simulation Tracking

ASTERIX tracks hundreds of aircraft simultaneously across a shared coordinate space. In Quantmind, the equivalent is running 10,000 Monte Carlo simulation paths in parallel across multiple correlated assets. Each simulation path is an independent "track" that evolves through time according to GBM dynamics. The Cholesky decomposition step is analogous to ASTERIX's multi-sensor data fusion — reconciling independently sampled signals into a coherent, correlated picture.

7.1.3 Data Encoding & Payload Efficiency

ASTERIX uses a compact verbose binary format to maximise throughput under constrained bandwidth. Quantmind applies the same principle to simulation result storage: the simulation_paths table stores paths as BYTEA (compressed Float64Array) rather than JSONB. API payloads for the mobile app are truncated to 200 sampled paths for display, with full data retained server-side — reducing mobile payload size by over 98% while preserving analytical fidelity.

7.1.4 Live Simulation Engine (Advanced Roadmap)

The most ambitious application of ASTERIX-inspired thinking is a continuously running simulation engine that reacts to live market data in near real-time — analogous to an ATC radar that refreshes the full airspace picture every few seconds. In this model, Quantmind would maintain a persistent simulation state for each active portfolio, re-running incremental GBM steps as new price data arrives and surfacing updated VaR and CVaR figures to the user without requiring manual re-simulation. This capability is scoped to a post-launch roadmap milestone and will leverage Supabase Realtime channels as the event bus.

7.2 Financial API Stack

Quantmind's simulation quality is directly proportional to the quality of its market data inputs. The platform targets a layered API stack: one provider for core historical and technical data, one for real-time streaming and live dashboards, and one for sentiment and fundamental overlays that enrich AI-driven analysis. All external API calls are proxied through Supabase Edge Functions — the mobile client never calls financial data APIs directly, ensuring key security and enabling server-side caching.

7.2.1 Alpha Vantage — Core Data Provider

Alpha Vantage serves as the primary data source for historical OHLCV prices, forex rates, crypto data, and built-in technical indicators including RSI and MACD. Its breadth makes it the ideal feed for populating the GBM engine's expected return (μ) and volatility (σ) parameters directly from observed historical data, reducing the burden on users to manually calibrate their simulations. The free tier provides approximately 25 requests per minute, which is sufficient for the assets/history Edge Function given server-side caching with a 1-hour TTL.

7.2.2 Twelve Data — Real-Time & WebSocket Feed

Twelve Data provides native WebSocket support alongside a clean REST API, making it the designated provider for live price streaming into Quantmind's continuous risk monitoring features. The WebSocket feed connects via the Supabase Edge Function layer, which normalises incoming ticks and publishes updates to Supabase Realtime channels. The mobile app subscribes to those channels to power live portfolio valuation and the real-time VaR recalculation pipeline described in section 7.1.1.

7.2.3 Finnhub — Sentiment & Fundamentals

Finnhub adds a qualitative intelligence layer to Quantmind's quantitative core. Its news sentiment scores and company fundamental data are injected into the AI context assembly pipeline in claudeService.ts, giving Claude models richer background when generating Portfolio Doctor analyses and stress test interpretations. For example, a high negative sentiment score on a held asset can prompt the AI to weight tail-risk scenarios more heavily in its narrative, producing more contextually relevant risk commentary for the user.

7.2.4 Supporting Providers

Yahoo Finance (via the yfinance Python library) supplements the primary stack as a fallback for historical price retrieval and options data, given its broad asset coverage and no formal rate limits on public endpoints. The World Bank API provides macroeconomic variables — GDP growth, inflation, and interest rate series — that feed into advanced simulation assumption presets, particularly for the goal-based planning module where long-horizon projections benefit from macro-calibrated drift parameters. Polygon.io is earmarked for the scaling phase: its institutional-grade tick data and aggregate endpoints will replace Yahoo Finance in the production simulation pipeline once the platform reaches sufficient volume to justify a paid tier.

7.3 API Integration Architecture

All financial data APIs are consumed exclusively through server-side Edge Functions. The mobile client and web app never hold API keys or issue direct requests to third-party financial data providers. This proxy architecture enables centralised caching (TanStack Query with 1-hour TTL for historical data), rate limit management, and transparent provider substitution without mobile app updates. The data flow is: Financial APIs → Supabase Edge Functions (normalise + cache) → Supabase Realtime / REST → React Native App.

The existing marketDataApi.ts service in the mobile app already implements this abstraction: ticker search is debounced at 300ms, historical prices are cached for 1 hour in TanStack Query, and correlation matrix computation is handled server-side by the simulation microservice using the same price series. The additions described in this section extend that foundation with a live streaming layer (Twelve Data WebSocket) and an enrichment layer (Finnhub sentiment) without breaking any existing contracts.

7.4 API Provider Reference

Provider	Role in Quantmind	Data Provided	Free Tier	Phase
Alpha Vantage	Core historical + technical data feed for GBM parameter calibration	Stocks, forex, crypto, RSI, MACD	25 req/min	Launch (MVP)
Twelve Data	Real-time WebSocket feed for live VaR recalculation and live dashboards	Real-time + historical stock prices, WebSocket	800 req/day	Launch (MVP)
Finnhub	Sentiment and fundamentals injected into AI context assembly	Real-time quotes, news sentiment, company fundamentals	60 req/min	Post-launch v1.1
Yahoo Finance (yfinance)	Fallback for historical prices + options data; currently used in	Historical OHLCV, options chains, financial statements	Unlimited (unofficial)	Current (existing)

Provider	Role in Quantmind	Data Provided	Free Tier	Phase
	assets-history Edge Function			
World Bank API	Macro drift calibration for long-horizon goal planning simulations	GDP, inflation, interest rates by country	Free, open	Post-launch v1.1
Polygon.io	Production-grade tick data to replace Yahoo Finance at scale	High-quality market data, aggregates, ticks	Limited free tier	Scale phase (paid)